B · WHERE

B．端云协同 Agent

从请求路由到步级协作、选择性披露与可信执行

A 决定何时并行　B 决定在哪里执行　C 决定执行什么
D 决定能否信任



端云矛盾不是二选一，而是多目标约束

- **左栏：**全本地
- **优点：**隐私、离线可用、无 API 费用、低网络依赖
- **缺点：**模型小、长轨迹脆弱、端侧 prefill/decode 慢
- **右栏：**全云端

- **优点：**推理和多模态能力强、工具生态成熟
- **缺点：**网络延迟、持续成本、原始 UI/ 本地数据暴露
- **底部结论：**协同的目标不是“尽量少用云”，而是在成功率约束下最小化云调用、延迟、通信与隐私暴露。

同叫“端云协同”，实际可能发生在四个不同层次

1 层次 1 任务层

云端分解，本地执行子任务（Minions）

2 层次 2 轨迹层

每个 Agent step 动态路由（Hera）

3 层次 3 表示层

只上传 UI block / feature / KV （CORE、split inference）

4 层次 4 执行层

本地过滤与守卫、云端语义判断（Cognitive Firewall、AgenTEE）

- **右下角提示：**比较论文前，必须先问“路由的单位是什么？”

B 方向代表论文：从协作协议走向可信 Agent runtime

- **时间线：** 2025 Minions → 2025 CORE → 2026 Mobile-Agent-v3.5 → 2026 Hera → 2026 Agent-X / AgenTEE / Cognitive Firewall
- **主讲种子：** Hera 、 CORE

- **补充桥梁：** Minions 、 Agent-X 、 Mobile-Agent-v3.5 、 AgenTEE 、 Cognitive Firewall
- **阅读问题：** 谁决定？传什么？何时上云？如何证明收益与安全？

Minions：让云端“指导”，而不是让云端读取全部本地上下文

- **问题：**本地小模型能读私有长文档，但复杂推理弱；云端强模型能力高，却不应接收全文。
- **Naive Minion：**本地与云端来回对话；远程成本降 30.4×，但仅恢复 87% cloud-only 性能。

- **失败原因：**本地模型难以执行复杂多步指令，也难以在长上下文中稳定推理。
- **转折：**把难任务拆成短、局部、可并行的子任务。

云端分解与聚合，本地在私有数据上并行执行

1 Step 1

云端基于任务生成结构化子任务。

2 Step 2

本地在不同文档 chunk 上并行执行短子任务。

3 Step 3

只把局部结果 / 证据返回云端，而非发送完整上下文。

4 Step 4

云端聚合、必要时验证并形成答案。

- **关键点：**协同收益来自“能力分工 + 数据最小化 + 并行执行”，不只是模型路由。

质量可接近云端，但它还不是长轨迹 Agent 路由器

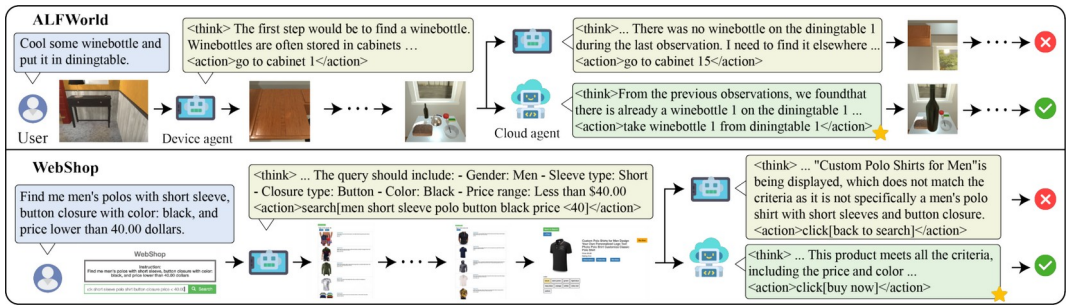
- **结果：** MinionS 平均降低 5.7× 远程成本，恢复 97.9% remote-only 性能。
- **优点：** 本地数据不离端；子任务并行；协议容易理解。

- **局限：** 面向长文档推理；环境基本静态；路由单位是子任务，不是 Agent 每一步。
- **过渡：** 当 Agent 与环境交互后，每一步难度都在变化——需要 state-level coordination 。

5.7×
远程成本降低

97.9%
cloud-only 性能

长轨迹中，云端价值集中在少数关键决策点



论文原图： Hera motivation

- **设备模型：**低延迟、低成本，但复杂规划和状态跟踪容易失败。
- **云端模型：**能力强，但每一步调用都会累积费用和网络等待。
- **论文观察：**设备失败而云成功的轨迹中，仍有大量步骤端云动作一致；理想 router 无需把整条轨迹交给云端。
- **难点：**某一步是否关键，与轨迹位置、推理长度、预测熵和环境状态共同相关。

在 cloud-only 性能约束下，最小化整条轨迹的云调用

- **动作混合：** â(s,x) = (1-rϕ(s,x)) · πd(s,x) + rϕ(s,x) · πc(s,x)
- **其中 rϕ(s,x)∈{0,1}：** 0 选择设备模型， 1 升级到云模型。
- **优化目标：** minϕ E[Σt rϕ(st,x)]

$$\hat{a}(s,x) = (1 - r_\phi(s,x))\,\pi_d(s,x) + r_\phi(s,x)\,\pi_c(s,x)$$

$$\min_{\phi} \; \mathbb{E}[\sum_{t} r_\phi(s_t,x)] \quad \text{s. t.} \quad \mathbb{E}[R(\tau_{mix})] \geq \mathbb{E}[R(\tau_{cloud})]$$

1 轨迹级约束

router 管理的不是单步答案质量，而是未来成功率与累计云成本。

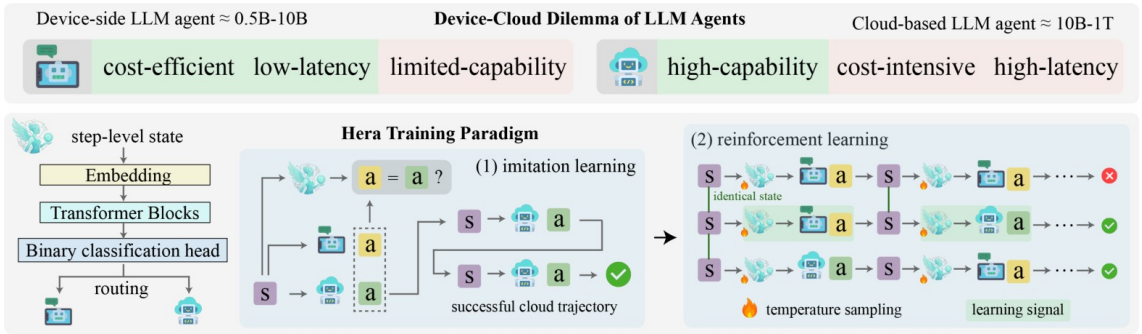
2 二元执行语义

rϕ=0 走设备， rϕ=1 升级云端；云端 policy 是质量锚点。

- **约束：** E[R(τmix)] ⩾ E[R(τcloud)]
- **解释：** router 不是估计单步“哪个回答更好”，而是在轨迹回报约束下管理未来云成本。

公式采用 Times New Roman / STIX，并作为独立透明 PNG 元素插入。

Stage I：用云轨迹回放构造 step-level 路由标签



论文原图： Hera IL stage

1 Step 1

分别 rollout device policy 与 cloud policy。

2 Step 2

选取 cloud 成功、device 明显较差的任务集合 Xdiff。

3 Step 3

在 cloud trajectory 的每个 state 上重放 device model。

4 Step 4

device action 与 cloud action 不一致 → 标注“需要云”；一致 → 标注“本地足够”。

5 Step 5

用 BCE 训练 500M 二分类 router。

优点：把离散路由转为可监督学习；缺点：动作相同只是 proxy，不直接等于最终成功。